

Stan Pierre
February 13 2026

Week 4 Report

Since the last report, I focused on improving player detection and getting a more reliable model running. I was able to fix the earlier compatibility problems by setting up a separate Python 3.11 environment and installing YOLOv8 from Ultralytics. That allowed me to move away from OpenCV's built in person detector and use a stronger object detection model.

What I actually ended up doing was getting the model to draw colored bounding boxes around detect players and then return coordinates for each one. Detection accuracy is noticeably better than before. It is consistently identifying players during pre snap frames. There are still issues. In some clips, grouping is off and the model occasionally includes referees or someone on the sideline. It also does not understand positions. Right now it only detects "person." I am storing the bounding boxes so I can begin figuring out spatial relationships between players instead of relying only on the detection.

The next step is to change from object detection to formation logic. I need to figure out the offensive line clutter and find the center, and then find all other player positions relative to that point. I also need to make the field orientation so formations can be shown consistently across clips. After that, I will work on making sure there is consistency in detection results and filtering out non players using field region limits.

Time Tracking

Week 4 (12 hours)

- Setting up Python 3.11 virtual environment and installing YOLOv8: 2 hours
- YOLOv8 into GridIQ: 3 hours
- Running detection on clips and messing with the model: 3.5 hours
- Setting up bounding box visualization: 2 hours
- Testing spatial grouping logic for offensive line: 1.5 hours

Stan Pierre
February 13 2026

Week 3 Report

Since the last report, I planned to do more research on OpenCV and switch over to Catapult's video system after getting logged out of our previous one. I also planned to use OpenCV on a small number of simple test cases to see how well it works at different scales.

What I actually ended up doing was running OpenCV on real football screenshots to test player detection. I set up a proof of concept using pre-trained models to see if it could identify players on the field. The results were mixed. It detected people pretty well, but it couldn't tell the difference between players, refs, coaches, and people on the sideline. It also couldn't distinguish between the two teams since it just sees everyone as "person." I ran into a technical issue with Python 3.13 compatibility, so I had to work around that by switching to OpenCV's built-in models instead of YOLOv8.

Moving forward, I need to figure out how to make the computer vision more accurate. Right now it's detecting too many people who aren't actually players on the field. I'm thinking about adding jersey color detection so it can tell the difference between the two teams. One team wears dark jerseys and the other wears white, so that should be doable. I also need to filter out refs, coaches, and sideline personnel. My idea is to focus on a specific area of the field where the actual play happens and ignore everything outside that zone. Once I can reliably identify just the players, I can start working on recognizing formations based on where they're positioned.

Time Tracking

Week 3 (11.5 hours)

- Setting up OpenCV environment: 2 hours
- Testing player detection on screenshots: 7 hours
- Troubleshooting Python compatibility issues: 1.5 hour
- Research on pre-snap frame extraction: 1 hour

Stan Pierre
February 6 2026

Week 2 Report

Since the last report, I planned to do more research on OpenCV and switch over to Catapult's video system after getting logged out of our previous one. I also planned to use OpenCV on a small number of simple test cases to see how well it works at different scales.

What I actually ended up doing was testing OpenCV on a few basic tasks, like having it tell the difference between photos I give it. I've got it set up in my project now. I also set up my accounts for the other systems I'll need to use going forward.

I'm planning to integrate video into the database somehow, since we need a place to store it. The problem is that video takes up a lot of space, so I need to figure out a way to keep storage low without spending money. The video itself is also pretty inconsistent, which makes identifying players harder. My idea right now is to grab a screenshot right before the ball is snapped. That's when everything is still and the positions are the clearest. From there, I can compare frames to see where players are between shots. This should be more accurate and could actually make tagging work.

Time tracking

Week 2 (5 hours)

- Downloading libraries : 1 hour
- Download videos: .5 hour
- using catapult framework: 1 hour
- Test OpenCv: 2.5 hours